

Designed by Remus Teodorescu 2026

LECTURE 1.1 · RECORDED

Python That Runs

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

COLAB · CONTAINERS · TRACEBACKS

ONE CELL

```
import numpy as np

def deflection(x, w, L, E, I):
    """Tip deflection of a cantilever, in m."""
    k = 6*L**2 - 4*L*x + x**2
    return w * x**2 * k / (24*E*I)

x = np.linspace(0.0, 2.0, 41)
y = deflection(x, 500.0, 2.0, 210e9, 4.5e-6)
print(y.shape, y[-1])
```

Every idea in this lecture is in these nine lines: an import, a function with a docstring, an array, a call, and a print that checks the shape before anybody believes the number. Then what to do when a cell stops.

Learning Objectives

SECTIONS, IN ORDER

1 **The notebook** — Colab, and why a cell is not a script

2 **The language** — values, containers, control flow, functions, imports

3 **Debugging** — the traceback, the six errors, and what to do in order

I assume you can already program. What you have probably not written is Python, and that gap is perhaps two hours of unfamiliarity. Watch this with a notebook open and type the cells as they appear.

READ, AND WHAT EACH ANSWERS

Python tut. 1	where does my code actually run?
Python tut. 3 – 6	which container, and what does it cost?
Python tut. 8	it stopped: what do I read first?

Colab

Colab — a Python notebook running on a rented machine in a data centre and shown to you in a browser tab. Everything this course needs is already installed and you need only a Google account. The machine is temporary: it is reclaimed after a period of inactivity, and every variable and file on it goes too.

A COMPUTER IN A BROWSER TAB, RENTED BY THE HOUR

The commonest reason a student loses the first week is an installation that will not work, and this removes it.

You need a Google account and nothing else. No install, no administrator rights on a university laptop, and the same environment for everybody, so when something goes wrong it goes wrong the same way for the person you ask.

TALK · 2 MIN In your own words, what is the difference between a notebook and the machine that runs it? Then, in the stack on the right, which of those five things survive the machine being reclaimed, and which do not?

WHAT LIVES WHERE

the notebook	in your Drive — permanent
the kernel	a Python process — restartable
variables	in that process — volatile
/content files	on the rented disk — lost
the GPU	if offered, and not needed yet

THE LOCAL ALTERNATIVE

Anaconda or a venv on your own laptop works identically. The Advanced Topics slide says how, once, and this course never mentions it again.

The Cell Model

The cell model — a notebook is not a script. Each cell is submitted to one long-lived Python process, the kernel, in whatever order you run them, and everything a cell defines stays in that process afterwards. The number beside a cell counts executions, not position, so it records the order that actually happened.

THE ORDER ON THE PAGE IS NOT THE ORDER OF EXECUTION

A script runs from the top to the bottom, once, in a fresh process. A notebook does not.

The number in the brackets to the left of a cell is not its position in the document. It is the count of how many cells the kernel has run, so it tells you the true order and it is the only thing on screen that does.

THE FAILURE, IN THREE CELLS

WHAT THE KERNEL BELIEVES

```
In [1]:  tau = 60.0
In [2]:  T = 22 + 58 * np.exp(-t / tau)

# you edit cell 1 to tau = 45.0
# and do not re-run it

In [3]:  print(tau)
         60.0
```

The screen shows forty-five. The kernel holds sixty. Nothing is wrong with your code and nothing raises an error — the number on the screen is simply not the number that produced the figure below it.

This has caused published errata. It is not a beginner's mistake.

Values and Types

Values and types — Python declares no types. A name is bound to a value and the value knows what it is. Four types carry almost everything in this course: int for counts and indices, float for measurements, str for text, and bool for the answer to a question.

NAMES ARE UNTYPED; VALUES ARE NOT

Ask a value what it is with the type function, which you should use far more often than pride suggests. There is no separate double: a Python float is already what MATLAB calls one.

The two division operators are the one genuine trap. A single slash always gives a float, even when both operands are whole numbers, and a double slash is floor division and gives an integer.

$$\sigma = \frac{F}{A}$$

σ the stress in Pa, F the force in N, A the area in m^2

Four lines of Python, on the right, and the docstring is not optional.

TYPES, AND THE DIVISION TRAP

```
n_samples = 31          # int
tau_s     = 60.0        # float
material  = 'aluminium' # str
converged = True        # bool

7 / 2   ->  3.5    (always a float)
7 // 2  ->  3      (floor division)
7 % 2   ->  1      (remainder)
2 ** 10 ->  1024   (power, not ^)
```

THE EQUATION ON THE LEFT

```
def axial_stress(force_N, area_m2):
    """Axial stress in Pa."""
    return force_N / area_m2
```

Lists and Slices

List — an ordered, mutable sequence written in square brackets, which may hold a mixture of types and can grow. Indices start at zero and negative indices count backwards. A slice is written start colon stop with the stop excluded, so the length of a slice is stop minus start.

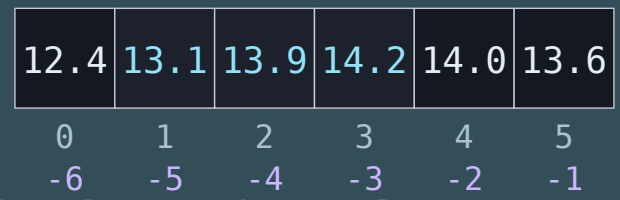
TWO CONVENTIONS THAT DIFFER FROM MATLAB

A name is not a box holding a value. It is a label attached to one, and two names can label the same list, so changing it through one changes it through the other. There is only one list.

When you want a copy, ask for a copy. This is the commonest surprise for an engineer arriving from MATLAB, where assignment copies and this one does not.

TALK · 2 MIN In your own words, why would a language make the end of a slice exclusive? Then, in the grid on the right, which elements does the slice one colon four return, and how many are there?

SIX ELEMENTS, AND WHERE INDICES SIT




```
y[1:4]  →  three elements, and 4 − 1 = 3
```

THE FOUR FORMS

```
y[0]    first        y[-1]   last
y[:3]   first three  y[3:]   the rest
y[::2]  every other  y[::-1] reversed
len(y)  6            y.append(v)
```

Dictionaries and Tuples

Dictionary — a container that maps keys to values, looked up by name rather than by position, which makes it the right one for anything where a number has a name: material properties, the settings of a run, a set of results. A tuple is its immutable cousin, and it is what a function returns when it returns several things at once.

THE RIGHT CONTAINER FOR A SET OF NAMED QUANTITIES

Two habits worth forming now. Iterate with the items method, which hands you the key and the value together, and use the get method with a default when a key might legitimately be absent.

A tuple is the immutable one, and it is what a function returns when it returns several things at once. That is why you can write two names on the left of an equals sign and have both filled in by one call.

A MATERIAL, AS A DICT

```
alu = {
    "name": "aluminium",
    "E_Pa": 69.0e9,
    "rho": 2700.0,
    "c_p": 900.0,
}

alu["E_Pa"]              69000000000.0
alu.get("k", 205.0)      205.0
"rho" in alu             True
list(alu)                the keys

for key, value in alu.items():
    print(f"{key:>6s}  {value}")
```

Indexing a key that is not there raises KeyError. That is a feature.

Control Flow

Control flow — a block in Python is defined by how far it is indented. There is no end keyword and there are no braces: the line introducing a block ends in a colon and the block is everything indented under it. Four spaces, always. A for loop iterates over the things themselves, not over a range of indices.

NO END, NO BRACES, AND NO ARGUING ABOUT IT

The universal convention is four spaces, and every editor you will use inserts them for you. Mixing tabs and spaces gives you an IndentationError, or worse, code that runs and means something other than it looks like.

The universal convention is four spaces, and every editor you will use in this course inserts them for you. Mixing tabs and spaces produces an IndentationError or, worse, a block that is not the block you meant.

THE THREE SHAPES

```
if strain > 0.002:
    verdict = "yielded"
elif strain > 0.001:
    verdict = "approaching yield"
else:
    verdict = "elastic"
```

```
for reading in gauges:
    total += reading

for i, reading in enumerate(gauges):
    print(f"gauge {i}: {reading:.2f}")
```

```
while residual > tol and step < 500:
    residual = iterate(residual)
    step += 1
```

Always give a while loop a second exit condition.

Comprehensions

List comprehension — a for loop that builds a list, written inside the brackets it produces, with an optional filter at the end. It is idiomatic Python, and it is not faster than the loop it replaces, so write one when it is clearer and not otherwise.

A LOOP THAT BUILDS A LIST, WRITTEN ON ONE LINE

You will read one in every library you touch, so you need to recognise it even if you choose not to write it.

It takes an optional filter — an if at the end — which is how you say the samples above the threshold in one clause instead of three lines.

THE SAME THING, TWICE

THE LOOP

```
squares = []
for v in readings:
    squares.append(v ** 2)
```

THE COMPREHENSION

```
squares = [v ** 2 for v in readings]

large = [v for v in readings if v > 3.0]

names = {k: v for k, v in alu.items()}
```

AND THE MEASUREMENT, NEXT LECTURE

Loop and comprehension differ by a few per cent. NumPy is faster by a factor in the tens to low hundreds.

Functions

Function — a named block introduced by def that takes arguments and returns a value. What belongs in the docstring is what the next person needs: what it does, what each argument is and in what unit, and what comes back. A mutable default argument is evaluated once and then shared between calls, which is the trap.

THE UNIT OF REUSE, AND THE UNIT OF TESTING

Define a function with def, a name, the arguments in brackets, and a colon. The body is indented.

Arguments can have defaults, and arguments can be passed by name at the call site. Passing by name is the right choice whenever a call has more than about three arguments, because a bare list of five numbers in brackets is unreadable six weeks later.

UNITS IN THE DOCSTRING, UNITS IN THE NAME

Ex_1_core writes force_N and area_m2 rather than F and A. It is slightly ugly and it has never once caused a unit error.

ONE FUNCTION, WRITTEN PROPERLY

```
def beam_deflection(x_m, load_N_per_m=500.0,
                    length_m=2.0):
    """Deflection of a cantilever, in metres.

    x_m is the distance from the fixed end.
    Uniform load, small-deflection theory.
    """
    L = length_m
    return (load_N_per_m * x_m**2 *
            (6*L**2 - 4*L*x_m + x_m**2)) /
(24*E*I)

y = beam_deflection(1.0, load_N_per_m=250.0)
```

The keyword at the call site is what makes the last line readable.

Imports and Namespaces

Import — bringing a name from another module into this one. Import a module under a short name and everything from it stays behind that name, which is what keeps the origin of a function visible where it is used. A bare star import puts every name into your namespace at once and hides it again.

THE PREFIX IS INFORMATION, NOT NOISE

An import binds a module to a name. Writing np dot linspace tells the reader that linspace comes from NumPy, and that is worth the four characters — particularly once torch is in the room and half the names exist twice.

The four aliases in this course are fixed and universal: np, plt, pd and torch, which is never abbreviated. Use them, because every example you find online uses them.

THE FIXED ALIASES

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import torch

from scipy.integrate import solve_ivp

from numpy import *   # never
```

THE SEARCH ORDER, AND ITS TRAP

Your own directory first, installed packages second. A file called numpy.py in your folder shadows the real NumPy, and the error you get will talk about a missing attribute rather than about your file.

The Traceback

Traceback — the report Python prints when an exception is raised: the chain of calls that led to it, innermost last, with the type and the message on the final line. It is not noise, it is the most useful diagnostic the language gives you, and it is read from the bottom upwards.

THE MOST SPECIFIC DIAGNOSTIC THERE IS

When something goes wrong, Python prints a traceback: the chain of function calls that led to the failure, oldest first, with the error itself at the very bottom.

Most people see a wall of red text, feel a small drop in confidence, and go back to staring at their own code. That is exactly backwards. The traceback already contains the answer. It has simply been printed in an order nobody explained.

WHY THIS IS THE SKILL TO HAVE

A student who reads tracebacks fluently solves in ninety seconds what otherwise costs an evening and a message to a classmate.

A REAL TRACEBACK, UNEDITED

```
Traceback (most recent call last):
  File "<ipython-input-7>", line 1, in <module>
    traceback_demo()
  File "Ex_1_core.py", line 511, in traceback_demo
    outer()
  File "Ex_1_core.py", line 502, in outer
    return middle([2.0, 4.0, 6.0])
  File "Ex_1_core.py", line 505, in middle
    return inner(values, 5)
  File "Ex_1_core.py", line 508, in inner
    return values[index]
IndexError: list index out of range
```

Twelve lines. Five of them are frames — a file, a line number, and the source line that was running. One of them is the error. The next two slides take this apart in the order that makes it useful.

Six Errors You Will Meet

The six errors — six exceptions account for most of what goes wrong in the first weeks, and each names one specific mistake. The one to fear is not on the list: an expression that raises nothing and prints a plausible wrong number.

EACH ONE MEANS ONE THING, AND THE MESSAGE SAYS WHICH

NameError
A name that does not exist yet. Usually a typo, or a cell you have not run — the kernel problem again.

TypeError
An operation between things that do not combine. A list plus a number; a None where a float was expected.

IndexError
An index outside a sequence. KeyError is its dictionary twin, and means the key is absent.

AttributeError
The object has no such method. Very often the object is None, because a function returned nothing.

AND THE ONE THAT DOES NOT RAISE AT ALL

Ex01's broken_average sums from index one instead of index zero and divides by the full length. It returns a number. The number is wrong. No traceback exists for this, and the only defence is checking one case you can compute in your head.

What To Do In Order

Debugging, in order — the order to work in when something breaks. Read the last line first. Find your own file in the trace. Print the values, the types and later the shapes at that point. Reduce the input until the error goes away. And only then change the code.

THE PROCEDURE, IN ORDER

Read the last line of the traceback and say out loud what it means. Then find the last frame that is your code. Then print the things on that line — the value, the type, and from next lecture the shape — before forming any theory at all.

The order matters. The commonest failure mode among competent engineers is to form a theory first and then look for evidence supporting it, which in a language you are new to is almost always the wrong theory.

IN ORDER, AND WITHOUT SKIPPING

1	read the last line, literally
2	find the last frame you wrote
3	print the value, the type, the shape
4	say what you expected, and why
5	change exactly one thing
6	shrink the case until you can check it
7	restart and run all, then believe it

PAUSE THE VIDEO HERE

Ex01 notebook 01, section six, asks you to diagnose a real traceback before reading the explanation. Do it in that order.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
virtual environments	one set of packages per project	Python tut. ch. 12	working off Colab, on your laptop
classes	state and behaviour in one object	Python tut. ch. 9	a solver you reuse, PyTorch modules
files and paths	reading and writing on disk	Python tut. ch. 7, 10	data that is not already loaded
testing	a check that runs itself	Python tut. ch. 26	code somebody else will trust
type hints	the argument types, written down	Python tut. ch. 4.9	long scripts, shared code

WHY THEY ARE LEFT OUT

None of them is needed to run a notebook, and every one of them is easier to learn once you have. Classes arrive by themselves in the PyTorch lecture, where you will subclass one before you have been told what a class is.

EXERCISE

Exercise 1

FOUR NOTEBOOKS, AND THE FIRST TWO ARE THIS LECTURE

Notebook 00 is a read-only environment check. Run it first and run it today, because a missing PyTorch found in week one costs nothing and the same discovery in week six costs an exercise session.

Notebook 01 is today's lecture as work you do rather than watch: types, containers, control flow, functions, imports, and a section on reading a traceback that is deliberately the longest in the notebook.

THE NOTEBOOKS

00	environment check — read only
01	today: types, functions, tracebacks
02	L1.2: arrays and broadcasting
03	L1.2: engineering figures

The running example is a two-metre steel cantilever under a distributed load, with a dial gauge good to about four hundredths of a millimetre. It appears in all four notebooks, so the shapes always mean something physical.

REPORT QUESTION, TO CONCLUDE

Across the notebooks: which error cost you the most time, what did its last line actually say, and what would have found it faster?

SUMMARY

Key Takeaways

THE LANGUAGE

what it is	names bound to values that know their own type
function	containers, control flow, functions, imports
train, tune	four spaces, zero-based, half-open slices
vs MATLAB	no end, no semicolons, and the slice excludes the stop
used for	every notebook in the twelve lectures after this

DEBUGGING

what it is	a traceback: the calls that led to the error
function	names the file, the line and the kind of mistake
train, tune	read the last line, then find your own file
vs guessing	evidence before theory, which is the whole habit
used for	every error you will meet for the rest of the course

Questions

1 – 2 · THE LANGUAGE

1 **How are names and values related in Python, and which containers carry the data in this course?**

A name is a label bound to a value, and the value knows its own type; assignment binds the name again and copies nothing. A list keeps an ordered sequence, a dictionary looks a value up by its key, and a tuple is a sequence that cannot change. *Python tut. ch. 3, 5; Ex_01 notebook 01*

2 **How does Python mark a block of code, and how does a slice count?**

By indentation: four spaces open a block and going back closes it; there is no end keyword. Counting starts at zero and a slice excludes its stop, so a[2:5] has stop minus start, three, elements. *Python tut. ch. 3 – 4; Ex_01 notebook 01*

3 – 4 · DEBUGGING

3 **What is a traceback, and how do you read one?**

The list of calls that led to the error, the most recent last. Read the last line first: it names the kind of mistake. Then go up to the first frame that is in your own file; that line is where to look. *Python tut. ch. 8; Ex_01 notebook 01*

4 **In what order do you work when something breaks, and which failure is the worst?**

Evidence before theory: read the message, print the value and its type, then change one thing. The worst failure raises nothing at all: the code runs and the number is wrong, so every result is checked against a case whose answer you know. *Python tut. ch. 8; Ex_01 notebook 01*

NEXT

L1.2 — NumPy, and the Shape of Everything

Today the language. Next time the object every later lecture is actually made of — the array — and one rule about shapes that explains most of the bugs you will hit in weeks seven to twelve.

BEFORE THE NEXT RECORDING

run	Ex01 notebook 00, all the way
work	Ex01 notebook 01 — about an hour
practise	reading one traceback aloud
do not	start notebook 02 yet

Notebook 02 is the broadcasting notebook and it is the one to protect if you run short of time. Watch L1.2 first; it exists to make that notebook take twenty minutes instead of two hours.